Modern Application Development 1 (MAD-1)
Project Report

1. Student Details

Name: Simran Padhy
Roll Number: 23f2002788
Email: 23f2002788@ds.study.iitm.ac.in

About Me: I am a student at IIT Madras Online BS Degree of Data Science and Applications. I have a strong interest in web application development, with a deep devotion to design and UI/UX. I am also pursuing a Dual Degree in Industrial Ceramics (Engineering) at NIT Rourkela. My personal goal is to combine these two fields to build data-driven digital solutions for the ceramics domain.

---

2. Project Details

Project Title: *PlaceMe* – Placement Portal Application

Problem Statement: Institutes rely on manual processes, spreadsheets, and emails to manage campus recruitment, making it difficult to handle company approvals, track student applications, prevent duplicate registrations, and maintain placement records.

Aim of the Project: To design and build a web-based application, i.e., a Placement Portal, to enable role-based interaction between the admin (Institute Placement Cell), companies and students. Its aim is to streamline campus recruitment activities. It will allow institutes to manage company approvals, track student applications, avoid duplicate registrations, maintain placement records, etc.

Approach: The application was built using Flask in a single file (*app.py*) that managed routes, models and setup. It used a three-role system, *Admin*, *Company*, and *Student*, with session-based access control. A dual-approval mechanism ensured companies must be approved (by the admin) and active, and jobs must also be approved by the admin before being visible. The placement process follows stages from Applied to Placed, with a placement record created at the final stage. A notification system informs students about status updates. A strict access control ensures companies only view relevant student profiles. Additionally, the system prevents duplicate job applications by allowing only one application per student per job.

---

3. AI/LLM Declaration

I used Claude (by Anthropic) to assist with reviewing SQLAlchemy model definitions, suggesting variable naming consistency, and generating documentation structure. The extent of AI/LLM usage is approximately 25–30%, limited to code review suggestions and documentation formatting. All core application logic, route design, model relationships, and debugging were done manually.

---

4. Technologies and Frameworks Used

Technology/Library	Version	Purpose
Flask	3.1.2	Core backend web framework such as routing, sessions and request handling.
SQLAlchemy	3.1.1	ORM for defining and querying the SQLite database.
Jinja2	Built in with Flask	Template engine for rendering dynamic HTML pages.
Bootstrap 5	5.3	Frontend styling and responsive layout.
Werkzeug	3.1.5	Password hashing (that is used to generate password hash and check password hash) and secure file upload.
SQLite	-	Lightweight local database. The file was created programmatically.

---

5. Database Schema / ER Diagram

The database consists of 7 tables. All tables and relationships are defined in *app.py* using SQLAlchemy models and created programmatically inside *init_db()*.

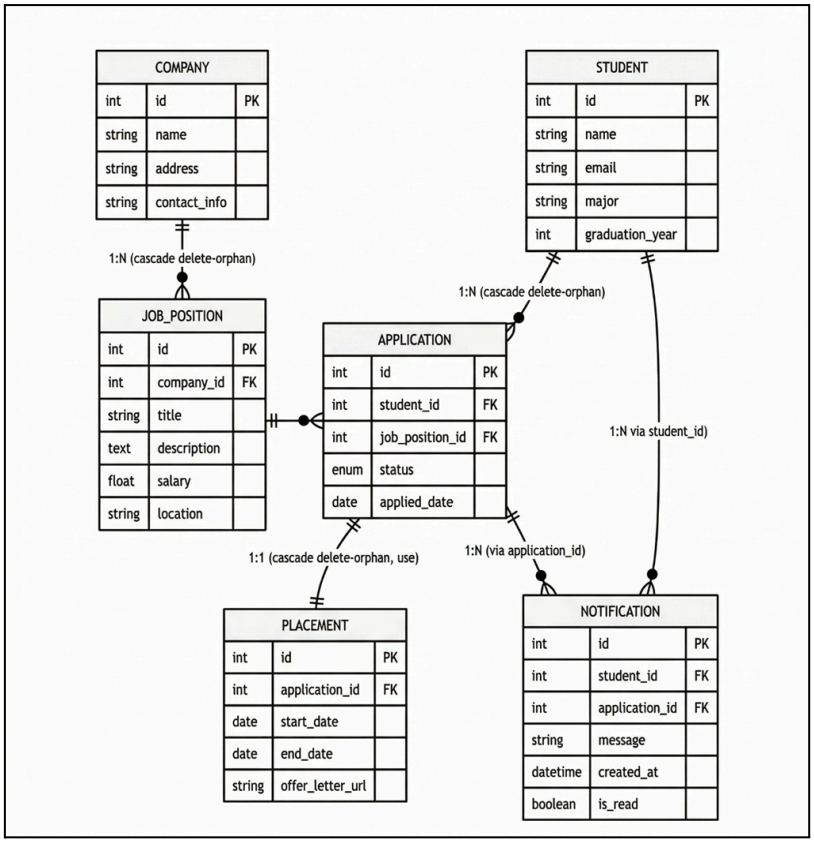
Tables and Key Fields with their Purpose:

Sl. No.	Tables	Key Fields	Purpose
1	Admin	*id, username, email, password* (hashed)	Single pre-seeded superuser; no registration route.
2	Company	*id, name, email, password, industry, contact, is_approved, is_active, registered_date*	Registered companies. Requires admin approval before login is permitted.
3	Student	*id, name, email, password, student_id, contact, education, skills, resume_path, is_active, registered_date*	Self-registered students. Resume uploaded as PDF/DOC/DOCX.
4	JobPosition	*id, company_id, title, description, required_skills, experience, salary_range, status, is_approved, posted_date*	Placement drives created by companies. Requires separate admin approval.
5	Application	*id, student_id , job_id, status, applied_date, updated_date*	Links students to jobs. Status flows: Applied → Shortlisted → Interview → Selected → Placed / Rejected.
6	Placement	*id, application_id , placement_date, joining_date, salary*	Auto-created when an Application status is set to Placed. One-to-one with Application.
7	Notification	*id, student_id , application_id , message, is_read, created_at*	Status-change alerts delivered to students. Unread count surfaced on the student dashboard.

Relationships:

- Company → JobPosition: One-to-Many
- Student → Application: One-to-Many
- JobPosition → Application: One-to-Many
- Application → Placement: One-to-One
- Student → Notification: One-to-Many via *student_id*
- Application → Notification: One-to-Many via *application_id*

ER-Diagram: Created using Mermaid.



---

6. Routes

The application exposes the following key routes, which are all session-protected and enforce role-based access.

Authentication:

Sl. No.	Endpoint	Method	Role	Description
1	/login	GET, POST	All	Role-based login; redirects to role dashboard on success
2	/register/<role>	GET, POST	Public	Registration for student or company only
3	/login	GET	All	Clears session

Admin Routes:

Sl. No.	Endpoint	Method	Description
1	/admin/dashboard	GET	Stats: total companies, students, jobs, applications, pending counts
2	/admin/companies	GET	List of all companies; supports search by name/industry
3	/admin/company/approve/<id>	GET, POST	Approve a company registration
4	/admin/company/reject/<id>	GET, POST	Reject a company registration
5	/admin/company/toggle/<id>	GET, POST	Activate or blacklist a company
6	/admin/students	GET	List of all students; supports search by name/ID/contact
7	/admin/student/toggle/<id>	GET, POST	Activate or blacklist a student
8	/admin/jobs	GET	View all placement drives
9	/admin/job/approve/<id>	GET	Approve a placement drive
10	/admin/job/reject/<id>	GET	Reject a placement drive
11	/admin/applications	GET	View all applications across all the students and companies
12	/admin/student/<student_id>	GET	View a student's full profile and application history
13	/admin/application/<id>	GET	View a single application in full detail

Company Routes:

Sl. No.	Endpoint	Method	Description
1	/company/dashboard	GET	Company stats: total jobs, active jobs, total applications
2	/company/jobs	GET	List own placement drives
3	/company/job/create	GET, POST	Create a new placement drive (pending admin approval)
4	/company/job/edit/<id>	GET, POST	Edit own placement drive
5	/company/applications/<job_id>	GET	View applications for a specific drive
6	/company/application/update/<id>/<status>	GET	Update application status (Shortlisted / Interview / Selected / Rejected / Placed)
7	/company/applications/all	GET	View all applications across all own drives, with status filter
8	/company/student/profile/<id>	GET	View shortlisted / selected student profile

Student Routes:

Sl. No.	Endpoint	Method	Description
1	/student/dashboard	GET	Stats: total applications, shortlisted, selected, rejected, placed + unread notifications
2	/student/jobs	GET	Browse all approved active drives; supports keyword search
3	/student/apply/<job_id>	GET	Apply to a placement drive (duplicate check + resume gate)
4	/student/applications	GET	View own applications with status summary
5	/student/applications/history	GET	Full application history timeline
6	/student/profile	GET, POST	View and update own profile; upload resume
7	/student/notifications	GET	View all notifications; marks all as read on open
8	/student/notifications/unread-count	GET	Returns JSON count of unread notifications

---

7. Architecture and Features

Project Structure:

- *app.py* — single entry point containing all models, routes, and Initialization.
- *templates* — Jinja2 HTML templates, one per view.
- *static/uploads/resumes/* — uploaded student resume files (PDF/DOC), stored with student ID prefix.

Additional Features:

- Search functionality for admin: students (by name, ID, contact) and companies (by name, industry).
- Company-side filter for applications by status across all drives.
- Context processor injecting student object into all student templates.
- Unread notification badge driven by a lightweight JSON endpoint.

---

8. Video Presentation

Drive Link: https://drive.google.com/file/d/1NBSg-pPjdcXUM-oPkJc3jzwLhA5PRerQ/view?usp=sharing

---